Project Report: Space Invaders (Pygame Edition)

Track Name: Game Development with Python (Pygame)

1. Introduction

This project is a functional 2D arcade shooter developed using the **Pygame** library. It features a structured game loop, asset management, and progression system. The game challenges players to defend against waves of enemies while managing power-ups and aiming for a completing levels.

2. Setup & Execution

- **Environment:** Python 3.13 with the pygame library installed.
- **Package Requirement :** `pygame`, **Matplotlib** ,Pillow .
- **Execution:** Run the script `space invaders.py`. The game initializes a 800x600 resolution window .

3. Core Gameplay Features

- **Movement & Combat:** The player controls a spaceship (64x64 pixels) using the Left and Right arrow keys. Pressing the **Spacebar** fires bullets to destroy enemies.
- **Enemy System:** Five enemies are spawned at random coordinates. They move across the screen and drop down whenever they hit a boundary.
- **Collision Detection:** The game uses a distance-based formula to determine hits between bullets and enemies, as well as player collisions with power-ups.
- **Score System:** Every successful hit increases the player's score. The current score and the "All-Time High Score" are displayed on the HUD.

4. Level & Progression System

The game features a dynamic state-management system that transitions players through different phases based on their performance:

- **Level 1:** The starting phase.
- **Level 2:** Triggered when the score reaches **100**.
- **Level 3:** Triggered when the score reaches **150**.
- **Victory State:** Reached when the player achieves a score of **300**.

5. Power-Up System

A **Rapid Fire** power-up (using the `powerup 32.`png asset) has a random chance (1 in 5) of dropping when an enemy is destroyed.

- **Mechanism:** Upon collection, the `rapid_fire` state is activated.
- **Duration:** The system uses `pygame.time.get_ticks()` to track time, allowing the player to fire at a much higher frequency for a set number of milliseconds before returning to normal speed.

6. UI/UX & Sound

- **Visuals:** The game uses `space.jpg` for the background and specialized sprites for the player (player64by64.png), bullets (bullets png.png), and aliens (alien type1 jpg).
- **Sound System:** Background music (`background.mp3`) loops continuously to provide an immersive atmosphere.
- **HUD:** A clear display of the current score and high score is rendered using Pygame's font module.
- **Main Menu**: we can select start or quit the game.
- **Pause & Quit**: When player presses **ESC**, the game state changes to paused.The pause only works in playing state.When paused players can choose to quit or resume the game.

7. Programming Concepts Used

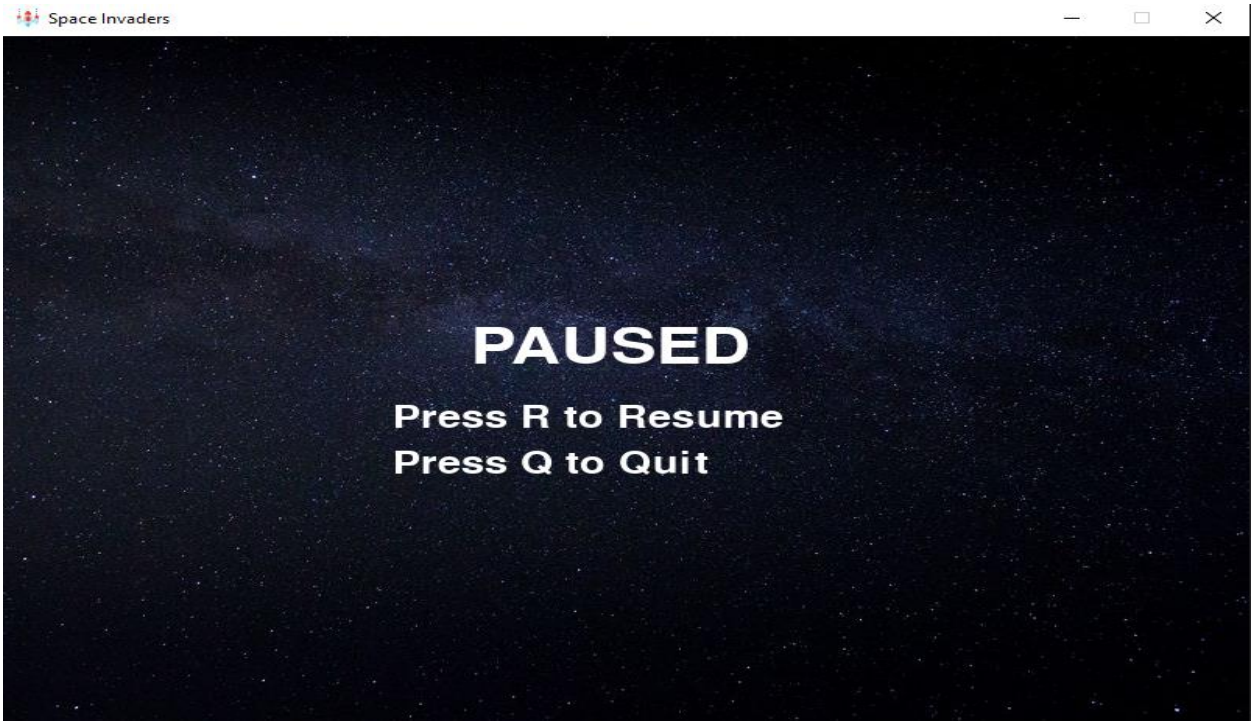
- **Game Loop & Event Handling:** A continuous `while` loop running at 60 FPS manages user inputs and screen updates.
- **Coordinate Geometry:** Precise X and Y coordinate manipulation for smooth movement and screen wrapping.
- **Randomization:** Used for enemy spawning and power-up drop rates to ensure every playthrough is unique.
- **State Machines:** Using a `state` variable to toggle between "running," "level_suggestion," and "victory" screens.

8. Conclusion

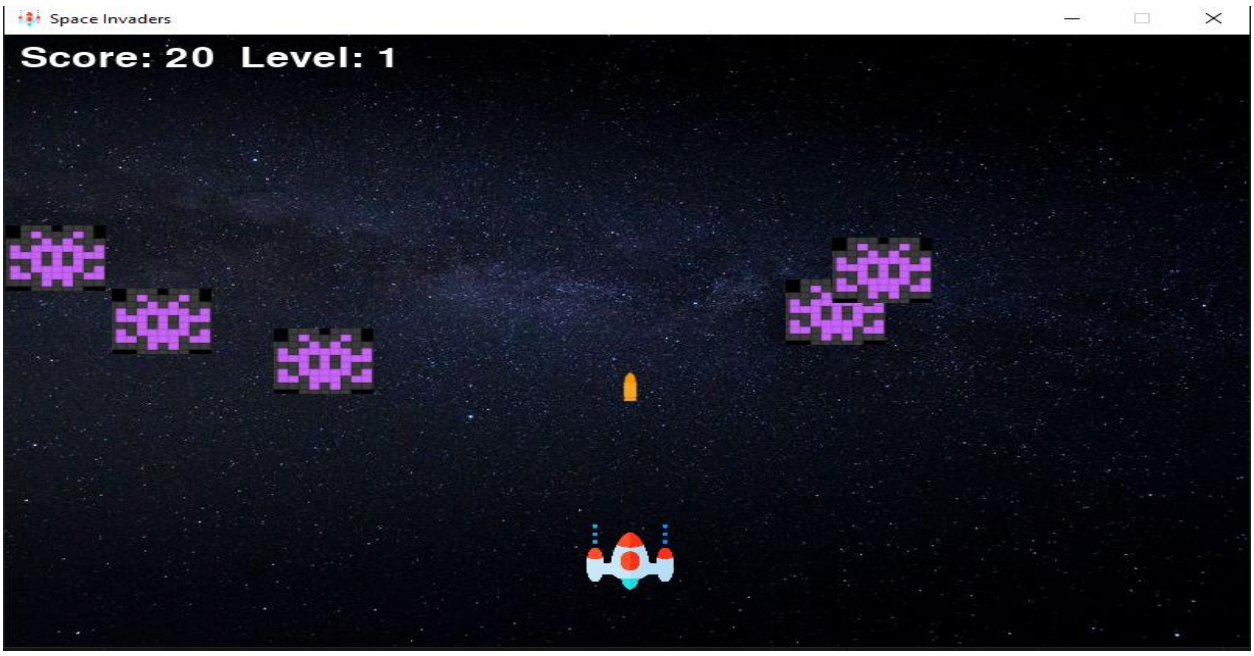
This project successfully demonstrates the core pillars of 2D game development. By integrating real-time collision math, timed power-ups, and persistent file saving, it serves as a complete example of how to build an engaging interactive experience using Python.

1:

2:


Space Invaders
PAUSED
Press R to Resume
Press Q to Quit

3:


Space Invaders
Score: 20 Level: 1

4:

Space Invaders
Score: 20 Level: 1
GAME OVER